

FastAPI Plant Disease Prediction Service

API Documentation

This document provides comprehensive details for the plant disease prediction microservice built with FastAPI.

Service Overview

This FastAPI service provides a machine learning endpoint to predict plant diseases from images. It leverages a pre-trained Keras model (`Plant_Disease_Detection.keras`) and integrates with the Gemini API to provide additional contextual information (description, causes, treatment, prevention) for detected diseases.

How to Run the Service

To set up and run your FastAPI application locally:

Prerequisites

- Python 3.8+
- pip (Python package installer)

Setup Steps

1. Model File Placement

Ensure your Keras model file (`Plant_Disease_Detection.keras`) is located in the same directory as your `main.py` (or `app.py`) script.

2. Install Dependencies

Open your terminal, navigate to your project's root directory (where `main.py` is located), and run:

```
bash
pip install fastapi uvicorn tensorflow pillow httpx
```

3. Set Gemini API Key (Optional but Recommended)

For the service to provide detailed descriptions, causes, treatment, and prevention, you need a Gemini API key.

- Obtain a key from [Google AI Studio](#)

- Open `main.py` and replace `"YOUR_ACTUAL_GEMINI_API_KEY_HERE"` with your actual key:

```
python  
  
GEMINI_API_KEY = "YOUR_ACTUAL_GEMINI_API_KEY_HERE"
```

Note: If the key is not set, the service will still predict the disease, but the contextual information will be placeholders.

4. Run the Application

Execute the following command in your terminal:

```
bash  
  
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Parameters:

- `main`: Refers to your `main.py` file
- `app`: Refers to the FastAPI instance named `app` within `main.py`
- `--reload`: Enables live reloading during development
- `--host 0.0.0.0`: Makes the server accessible from your network
- `--port 8000`: Specifies the port (you can change this if needed)

The service will be accessible at `http://127.0.0.1:8000` or `http://localhost:8000`.

API Endpoints

1. Plant Disease Prediction

Endpoint: `POST /predict`

Description: Receives an image file, preprocesses it, and returns the predicted plant disease, confidence level, and AI-generated information (description, causes, treatment, and prevention).

Parameters (multipart/form-data)

Send your image file as a multipart/form-data request. The image should be sent under the field name `file`.

Key	Type	Required	Notes
file	Image File	Yes	The image of the plant leaf (e.g., JPG, PNG)

Response Examples

Disease Detected (200 OK)

```
json
{
  "disease": "Apple Scab",
  "confidence": 98.75,
  "description": "Apple Scab is a common fungal disease affecting apple trees, causing olive-green to brown spots on lea",
  "causes": [
    "Fungus spores",
    "Wet conditions",
    "Infected debris",
    "Poor air circulation",
    "Susceptible varieties"
  ],
  "treatment": [
    "Apply fungicides early",
    "Prune infected branches",
    "Remove fallen leaves",
    "Improve air flow",
    "Plant resistant varieties"
  ],
  "prevention": [
    "Choose resistant cultivars",
    "Sanitize pruning tools",
    "Clear fallen leaves",
    "Proper tree spacing",
    "Avoid overhead watering"
  ]
}
```

Healthy Plant Detected (200 OK)

```
json
```

```
{
  "disease": "Apple healthy",
  "confidence": 99.50,
  "description": "Perfectly healthy plant. Apple trees are deciduous, known for their crisp, edible fruit. They thrive in temperate climates.",
  "causes": [
    "No specific disease causes."
  ],
  "treatment": [
    "No treatment needed. Continue good plant care."
  ],
  "prevention": [
    "Maintain optimal growing conditions, proper watering, and nutrition."
  ]
}
```

Background Without Leaves Detected (200 OK)

```
json
{
  "disease": "Background without leaves",
  "confidence": 99.12,
  "description": "No leaves found in the image. Please upload an image containing plant leaves for analysis.",
  "causes": [
    "Image does not contain leaves."
  ],
  "treatment": [
    "No treatment applicable as no leaves were detected."
  ],
  "prevention": [
    "Ensure images contain clear plant leaves."
  ]
}
```

2. Health Check

Endpoint: GET /health

Description: Checks the operational status of the FastAPI service and confirms whether the machine learning model has been successfully loaded into memory. This endpoint is useful for monitoring and readiness probes in deployment environments.

Parameters

This endpoint does not require any parameters.

Response (200 OK)

```
json
{
  "status": "ok",
  "model_loaded": true
}
```

Response Fields:

- `status`: Will be "ok" if the FastAPI application is running
- `model_loaded`: Will be `true` if the Keras model was loaded successfully on startup; `false` otherwise

Error Responses

If an error occurs during API processing, the service will return a JSON response with an appropriate HTTP status code and a detail message.

Example Error Output

```
json
{
  "detail": "Error message describing the issue."
}
```

Common Error Scenarios:

- **400 Bad Request:** Invalid input (e.g., no file uploaded, unsupported file format)
- **500 Internal Server Error:** An unexpected error occurred on the server side during prediction or content generation
- **503 Service Unavailable:** The ML model is not loaded (e.g., during application startup or if loading failed)

For additional support or questions, please refer to the [FastAPI documentation](#) or contact the development team.